



QUERY DATA WITH DYNAMODB

By Nabil Ahmed | September 2026

Query Data with DynamoDB

```
nextworksampldata $ aws dynamodb get-item \  
> --table-name ContentCatalog \  
> --key '{"Id":{"N":"202"}}' \  
> --projection-expression "Title, ContentType, Services" \  
> --return-consumed-capacity TOTAL  
{  
  "Item": {  
    "ContentType": {  
      "S": "Video"  
    },  
    "Title": {  
      "S": "Don't miss out!"  
    }  
  },  
  "ConsumedCapacity": {  
    "TableName": "ContentCatalog",  
    "CapacityUnits": 0.5  
  }  
}
```

Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a non-relational database service. DynamoDB organises data into tables, where data is organised using items and attributes.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to create and load tables, run queries over the AWS CLI and console. I also processed transactions (i.e. ran multiple operations as one) to manage related data.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is that DynamoDB queries have to use the partition key. In scenarios where I want to find data using only other attributes, I ran into an error.

This project took me...

This project took me me 1 hour including documenting my learning and understanding what was going on.

Querying DynamoDB Tables

A partition key is the main tag/id that DynamoDB will use to partition its tables' items. Every item must have a partition key, the partition key's value must be unique unless a table also has a sort key.

A sort key is a second filter/tag that DynamoDB can use to search for specific items. If a table has a sort key, then each item's primary key is made of its partition key and sort key. This means partition keys can be the same if the item has a different sort key.

▼ Scan, query or search items [Info](#)

☐ Scan

☒ Query

☐ Search

Select a table or index

Select attribute projection

Table - Comment ▼

All attributes

Partition key

Attribute

Value

Id

I have a question/Just Complete Project #7 Dependencies and CodeArtifacts

Sort key - optional

Attribute

Value

CommentDateTime

Greater than ▼

2024-09-01

☐ Sort descending

Limits of Using DynamoDB

I ran into an error when I queried for all comments posted by a specific user. This was because queries MUST use the table's partition key and the PostedBy attribute (which I used to filter the results) was not the partition key.

Insights we could extract from our Comment table includes all comments left on a single post. Insights we can't easily extract from the Comment table includes all comments made by a specific user.

▼ Scan, query or search items [info](#)

☐ Scan

☒ Query

☐ Search

Select a table or index

Table - Comment

Select attribute projection

All attributes

Partition key

Attribute

Id

Value

Enter attribute value

The partition key filter cannot be empty.

Sort key - optional

Attribute

CommentDateTime

Value

Greater than

Enter attribute value

☐ Sort descending

▼ Filters - optional

Attribute name

PostedBy

×

Condition

Equal to

Type

String

Value

User Abdulrahman

Remove

Add filter

Run

Reset

Running Queries with CLI

A query I ran in CloudShell was `aws-dynamodb-get-item`. This query will grab the item from the table and with the partition key value that I described in the command.

Query options I could add to my query are `consistent-read` (which provides a strongly consistent read), `projection-expression` (i.e. only return certain attributes) and `return-consumed-capacity` (which returns the number of capacity units consumed).

```
nextworksampldata $ aws dynamodb get-item \  
> --table-name ContentCatalog \  
> --key '{"Id":{"N":"202"}}' \  
> --projection-expression "Title, ContentType, Services" \  
> --return-consumed-capacity TOTAL \  
{  
  "Item": {  
    "ContentType": {  
      "S": "Video"  
    },  
    "Title": {  
      "S": "Don't miss out!"  
    }  
  },  
  "ConsumedCapacity": {  
    "TableName": "ContentCatalog",  
    "CapacityUnits": 0.5  
  }  
}
```

Transactions

A transaction is a group of operations that are bundled together and get executed as one. For the transaction to succeed, all operations within that transaction must process successfully.

I ran a transaction using AWS CLI commands ran on AWS CloudShell. This transaction did two things, first it added a new item to a table (Comment); then the transaction updated an item in the forum table (Event's Comment count increased by 1).

```
nextworksampladata $ aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[
> {
>   "Put": {
>     "TableName" : "Comment",
>     "Item" : {
>       "Id" : {"S": "Events/Do a Project Together - NextWork Study Session"},
>       "CommentDateTime" : {"S": "2024-9-27T17:47:30Z"},
>       "Comment" : {"S": "Excited to attend!"},
>       "PostedBy" : {"S": "User Connor"}
>     }
>   },
>   {
>     "Update": {
>       "TableName" : "Forum",
>       "Key" : {"Name" : {"S": "Events"}},
>       "UpdateExpression": "ADD Comments :inc",
>       "ExpressionAttributeValues" : { ":inc": {"N" : "1"} }
>     }
>   }
> ]'
```